Fundamentals of Data Engineer

🚀 What is Data Engineering?

Why Data?

- All major internet businesses (Amazon, Netflix, Zomato) rely on understanding their data to:
  - Improve customer service
  - Increase profit
  - Find business bottlenecks
  - Continuously improve their process
- Moving from assumptions to **data-driven systems** allows decisions that are factual, reliable, and scalable.paste.txt

Business Pipeline

1. User interacts with an application.
2. Every action generates data (click, like, purchase, etc.).
3. Data is stored in a **DBMS** (Database Management System).
4. Data Engineering, Data Science, Machine Learning, and Analytics operate on this data to provide insights or automate tasks.
5. Final outputs can be dashboards, recommendations, or predictive systems.paste.txt

---

🛠️ Roles and Responsibilities

- **Software Engineers:** Build applications (web/mobile), write and deploy code.

- **DBA (Database Administrators):** Design and manage databases, tables, columns.
- **Data Engineers:**
  - Build ETL pipelines (Extract, Transform, Load)
  - Design databases and data warehouses
  - Implement big data processing (e.g., with Spark/Hadoop)
  - Integrate data from APIs, sensors, RDBMS, etc.
  - Ensure data quality and governance
- **Data Scientists / Analysts:**
  - Analysts answer “what happened” using historic data
  - Scientists predict “what will happen” (forecasting)
- **Machine Learning Engineers:** Build, deploy, and maintain recommendation or predictive models.paste.txt

---

🔁 Data Engineering Lifecycle

> Book Reference: Fundamentals of Data Engineering

Key Steps:

- Data Generation (APIs, RDBMS, Sensors, Analytics)
- Ingestion (aggregating/inserting into systems)
- Storage (store/organize, e.g., in databases)
- Transformation (data cleaning, restructuring, logic)
- Serving/Consumption (ML, Analytics, Dashboards)

Diagram:

Application → DBMS → Ingestion → Storage → Transformation → Serving (dashboard/ML/model/report)

Transformation Examples:

- Standardize date formats
- Remove duplicates/nulls

- Aggregation/joins/calculated columns
- Done with Python, Spark, SQL, etc.paste.txt

🗄️ Databases and Data Modeling

Database Management Systems (DBMS)

- Designed to store/query structured data at scale
- Examples:
  - PostgreSQL, MySQL (Open Source)
  - SQL Server, Oracle (Enterprise)paste.txt

CRUD Operations:

```
`sql*-- SELECT query example*
SELECT * FROM student WHERE id=2;

*-- INSERT*
INSERT INTO student (id, name, age, city) VALUES (3, 'Akash', 25, 'Delhi');

*-- UPDATE*
UPDATE student SET age=27 WHERE id=1;

*-- DELETE*
DELETE FROM student WHERE id=3;`
```

Data Modeling

- Visual/logical design of tables and relationships.
- Example: Amazon's data model might include:
  - Orders, Users, Products, Payments, Shipping, Sellers

Relationships Example:

Table	Relationship	Key
Orders	Has user, product, payment	user_id, product_id, payment_id (FKs)
Products	Has seller	seller_id

Table	Relationship	Key
Shipping	Refers to order	order_id

- Data modeling tools: **draw.io**, DB designer appspaste.txt

---

🗃️ SQL vs NoSQL

Feature	SQL (Relational)	NoSQL
Data Structure	Tables, rows & columns, Schema	Key-Value, Document, Column, Graph
Schema	Fixed	Flexible
Query Language	SQL	Proprietary/No SQL
Data Integrity	Strong (ACID)	Varies
Use cases	Transactions, analytics	Large-scale scalability, flexible schema paste.txt

---

⚡ OLTP vs OLAP

OLTP	OLAP
Online Transaction Processing	Online Analytical Processing
Optimized for: Insert/Update	Optimized for: Aggregates, Analytics
Storage: Row-based	Storage: Column-based
Example: E-commerce checkout	Example: Sales reports
Technology: RDBMS	Technology: Data Warehouse

- Data Engineers often build pipelines to move data from OLTP → OLAP, applying transformations on the way.

🔄 ETL Pipeline: Extract, Transform, Load

Extract: Retrieve raw data from source systems.

Transform: Clean, standardize, join, filter (apply business logic, e.g., unify date formats).

Load: Store processed data in target (data warehouse, data lake, etc.).

- ETL is usually implemented in Python, Spark, or tools like AWS Glue.
- ETL may contrast with **ELT** (Extract, Load, THEN Transform in the warehouse).paste.txt

---

📊 Data Warehousing and Dimensional Modeling

Concepts

- Analytical workloads need data in a format supporting fast aggregation.
- **Fact Table:** Quantitative business data (sales amount, quantity, etc.)
- **Dimension Tables:** Descriptive data (products, users, dates, etc.)

Star Schema: Fact table at center, connected via FKs to dimension tables (star-like structure)

Snowflake Schema: Normalized version where dimensions can have sub-dimensionspaste.txt

---

🔃 Slowly Changing Dimensions (SCD)

- Dimensions (descriptive info) can change; SCD methods manage history.

SCD Type	Description	Example
1	Overwrite	City changes: NY → NJ. Old value gone.
2	Add Row	New row for each change, tracks full history
3	Partial	Store current & previous value in two separate cols
6	Hybrid	Combine multiple SCD types

🗃️ Data Marts

- **Data Marts:** Department/team-specific subsets of the data warehouse (e.g., just shipping or payment info).

🗂️ Data Lake vs Data Warehouse

Feature	Data Warehouse	Data Lake
Data Type	Structured (schema-on-write)	Raw/unstructured (schema-on-read)

Feature	Data Warehouse	Data Lake
Storage	Tables, SQL	Object storage (S3, Blob, GCS)
Use Cases	BI, Reporting, Dashboards	ML, Stream, Ad hoc analytics
Flexibility	Less (schema enforced)	High (any file format)

☁️ Cloud Computing and Services

- Cloud providers: **AWS, Azure, Google Cloud**.
- **PaaS:** Platform as a Service (e.g., AWS Lambda)
- **SaaS:** Software as a Service (e.g., Google Suite)
- **IaaS:** Infrastructure as a Service (e.g., EC2, EMR)

Service Examples (AWS):

- EC2 (VMs), Lambda (serverless compute), EMR (Spark/Hadoop), S3 (object storage/data lake), Redshift (data warehouse), Glue (serverless ETL), RDS (managed SQL database), DynamoDB (NoSQL), Kinesis (real-time streaming).paste.txt

Cloud Service Recommendation by Use Case:

- Large enterprises: Azure (good Office integration)
- Startups: AWS (more credits, ecosystem)
- Analytics-focused: BigQuery (GCP)
- Hybrid: Mix services from different clouds as needed

🧩 Real-World Architectures

> Example: Dream11 on AWS

- Data from apps/DBs/APIs → Kafka → S3 data lake → Spark ETL → Redshift warehouse
- Real-time needs: Kafka/Flink → Elasticsearch
- Ad hoc queries: Athena from data lake
- BI: Looker dashboards

- Orchestration: Airflow/Glue
- ML: SageMaker, Jupyter
- Data pipeline covers batch and streaming use casespaste.txt

> GCP/Azure Equivalents

- GCP: Cloud Storage, DataProc, BigQuery, Pub/Sub, DataFlow, Looker
- Azure: Data Lake Gen2, Data Factory, Databricks, Synapse Analytics, PowerBI

⚡ Modern Data Stack

Layer	Modern Tool Example
Ingestion	Airbyte, Fivetran, Stitch
Storage	BigQuery, Snowflake
Processing	Databricks (Spark), Glue
Transformation	DBT
Orchestration	Airflow, Prefect, Mage, Dagster
BI/Visualization	Looker, Data Studio
Data Quality	Great Expectations
Metadata	OpenLineage, DataHub

- "Reverse ETL": Transform data, then feed it back to source systems.paste.txt

---

🔧 Essential Skills and Tools

Skill/Tool	Why Important / What to Learn
Python	Data wrangling, ETL, file formats
SQL	Query databases, build data models
Linux Commands	80-90% of servers run Linux
Data Warehouses	Snowflake, BigQuery, Redshift
Big Data	Apache Spark, Kafka, Flink
Orchestration	Apache Airflow, Mage, Prefect

Skill/Tool	Why Important / What to Learn
Data Quality	Great Expectations
Modern Tools	DBT, Dagster, Mage, DataHub

Mini Python Example:

```
pythonimport pandas as pd

# Load CSV and clean data
df = pd.read_csv('data.csv')
df['date'] = pd.to_datetime(df['date'], format='%Y-%m-%d')
df.drop_duplicates(inplace=True)
```

🔒 Data Security & Masking

- **Confidentiality** (only authorized access)
- **Integrity** (correct/complete data)
- **Availability** (accessible to approved users)
- **Data Masking:** Mask fields for privacy (ex: Social security → XXX-XX-1234)
- **File Formats:** JSON, CSV, Parquet, ORC used in pipelines

---

📚 How to Build Your Learning Path

Start with:

- Python: variables, data structures, file formats, Pandas, datetime
- SQL: CRUD, DDL/DML, joins, aggregation, window/analytical functions, data modeling, constraints
- Data modeling/project: E-commerce or social media as sample use case
- Warehouse: OLTP/OLAP distinction, design, tools (Snowflake/BigQuery/Redshift)
- Spark: Architecture, DataFrames, transformations/actions, RDDs, distributed processing
- Orchestration: Airflow basics and building DAGs

🔗 Resources, Courses, and Projects

- Courses available on DataVidhya and YouTube (hands-on, project-driven)
- Projects in Python, Snowflake, Spark, Airflow, real-time streaming, GCP, Azure
- All courses include interactive notes, templates, and access to Discord community

---

Other Topics Covered

- Choosing the right cloud, architecture flexibility
- Hybrid/multi-cloud approaches
- Data file formats and their tradeoffs (schema-on-read vs schema-on-write)
- Design patterns in ETL pipelines for reliability and maintainability

---